Team: Daniel Brown, Noah Dougherty, Tyler Baker, William Schoennberger

A. Hierarchy

We split the program into a few main parts. Each function does one job, then the functions are put together at the end.

1. **Get valid letters** -
   Find the ten unique letters used in the puzzle.
2. **Check the guess** -
   Make sure the user's guess contains exactly ten unique letters from the puzzle.
3. **Check the equation** -
   Determine whether the dividend equals the quotient multiplied by the divisor plus the remainder.
4. **Convert words to numbers** -
   Use the user's guess to assign each letter a number from 0 to 9.
5. **Make the numbers** -
   Convert the dividend, quotient, divisor, and remainder into integers using the letter-to-number mapping.
6. **Run the program** -
   Ask the user for a puzzle and one guess, print the puzzle, validate the guess, and display the appropriate message.

B. Variables

Name	What it is for
puzzle	Stores the word arithmetic puzzle entered by the user.
valid_letters	Stores the ten unique letters found in the puzzle.
guess	Stores the user's guess for the letter-to-number mapping.
dividend	Stores the integer value of the dividend word.
quotient	Stores the integer value of the quotient word.
divisor	Stores the integer value of the divisor word.

remainder	Stores the integer value of the remainder word.
numbers	Stores the tuple of four integers returned by make_numbers.
letter	Represents an individual letter when processing the puzzle or guess.
word	Stores a word from the puzzle that needs to be converted into a number.
number	Stores the integer value of a word after conversion.
letter_to_number	A dictionary that maps each letter in the guess to its corresponding digit from 0 to 9.

C. Test Cases

Some of the test cases we can try after the code is put together are:

Test 1

Input: Puzzle = RUE,EAR | RUMORS,UEII ,UKTR ,EAR ,KEOS,KAIK,USA,

guess = TAKEOURSIM.

Expected: The program prints the puzzle, accepts the guess as valid, converts the words into integers, and outputs Try again! because the guess does not solve the equation.

Test 2

Input: The same puzzle,

guess = TAKEOUR.

Expected: The program prints the puzzle and outputs Your guess should contain exactly 10 unique letters used in the puzzle. Because the guess does not contain ten unique letters.

Test 3

Input: The same puzzle, guess = a correct solution containing exactly ten unique letters.

Expected: The program prints the puzzle, converts the words into integers, checks the equation, and outputs Good job!.

Test 4

Input: The same puzzle,

guess = IMAKEIMAKE.

Expected: The program prints the puzzle and outputs Your guess should contain exactly 10 unique letters used in the puzzle. Because the guess contains repeated letters.

Test 5

Input: A valid word arithmetic puzzle and a guess containing ten unique letters from that puzzle that does not satisfy the equation.

Expected: The program accepts the guess as valid but outputs Try again! because the arithmetic equation is incorrect.

G. Team Reflection

1. Describe the difficulty your team had when combining the code at the end. How could your team have specified the design more clearly at the beginning?

Combining the code was difficult because everyone worked on different parts, and we had to make sure they all worked together effectively. Some functions depended on the results of other functions, so we had to make sure they used the correct inputs and returned the expected values. We also had to ensure that the letter-to-number conversion and equation checking worked correctly together. Next time, we could agree on function names, parameters, return values, and how the code would be organized before splitting up the work.

2. Describe any benefits and drawbacks you saw by dividing the coding like this. Can you see reasons why this might be a good idea? How about a bad idea?

Splitting up the work helped because everyone could focus on one function and work on different parts of the program at the same time. This made the assignment easier to manage as a group. However, combining the code was harder because each function had to work correctly with the others. If people used different assumptions about inputs,

outputs, or variable names, it could cause errors. To make this easier, we should communicate throughout the project, agree on the program's design beforehand, and test the complete program together.